

Catena programmata

Esercitazione 5
Architettura degli elaboratori

Esercizio 1

Dato il seguente frammento di codice:

```
.data 0x10010000
```

```
ciao:
```

```
    .ascii "Ciao Ciao."
```

```
mamma:
```

```
    .ascii "Mamma"
```

```
t:
```

```
    .space 6
```

```
eol:
```

```
    .ascii "1"
```

Quale valore assume l'etichetta "t"?

Quale valore assume l'etichetta "eol"?

N.B. assembly directives pag. A-47/A-48

Esercizio 1 – soluzione

Dato il seguente frammento di codice:

```
.data 0x10010000
```

```
ciao:
```

```
    .ascii "Ciao Ciao."
```

```
mamma:
```

```
    .ascii "Mamma"
```

```
t:
```

```
    .space 6
```

```
eol:
```

```
    .ascii "1"
```

Quale valore assume l'etichetta "t"?

L'etichetta "t" assume valore 0x10010010, ovvero l'indirizzo 10+1+5 (=16) byte dopo l'inizio del segmento data (che inizia dall'indirizzo 0x10010000).

Infatti la direttiva ".ascii" aggiunge anche il carattere di fine stringa mentre la direttiva ".ascii" no

Quale valore assume l'etichetta "eol"?

L'etichetta eol si trova 6 byte dopo l'etichetta t, ovvero all'indirizzo 0x10010016

Esercizio 2

- Cosa sono le pseudoistruzioni e come si comporta un assemblatore quando ne incontra una?

Esercizio 2 – soluzione

- Cosa sono le pseudoistruzioni e come si comporta un assemblatore quando ne incontra una?
- Le pseudoistruzioni sono istruzioni non implementate dall'hardware (cioè, non native dell'ISA di MIPS)
- Sono tradotte dall'assemblatore in istruzioni native o sequenze di istruzioni native (secondo le convenzione d'uso dei registri, il registro \$1 - \$at è riservato a questo scopo)

Esercizio 3

Quali direttive assembly permettono di definire un'etichetta “str” in modo che possa essere visibile da altri moduli del programma?

In quale porzione della memoria sarà salvata str?

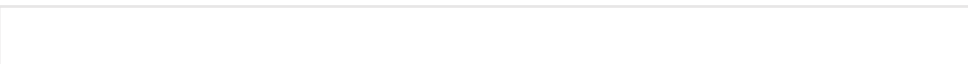
Esercizio 3 – soluzione

Le direttive assembly che permettono di definire str in modo che possa essere visibile da altri moduli del programma sono:

- `.globl str` `#visibile all'esterno`
- `.extern str 16` `#visibile all'esterno e di 16 byte`

La porzione della memoria in cui sarà salvata str è quella riservata ai dati statici

N.B. il simulatore QTSPIM potrebbe memorizzare i dati statici a partire da un indirizzo diverso (e.g. 0x10010000) da quello indicato dal libro di testo (pag. A-21) che si riferisce a MIPS

 MIPS systems typically dedicate a register (`$gp`) as a *global pointer* to the static data segment. This register contains address 10008000_{hex}, 

Esercizio 4

Dato il seguente programma

.data 0x10010000

.text

#qui ci sono 2 istruzioni, nessuna pseudoistruzione e non ci sono definizioni o uso di label

j primaLabel

secondaLabel:

#qui ci sono 3 istruzioni, nessuna pseudoistruzione e non ci sono definizioni o uso di label

terzaLabel:

#qui ci sono 4 istruzioni, nessuna pseudoistruzione e non ci sono definizioni o uso di label

j secondaLabel

A quante delle label presenti, l'assemblatore assocerà correttamente un indirizzo?

Quante e quali label saranno “unresolved” dopo l’assemblaggio?

Se al simbolo secondaLabel è associato il valore 0x00400030, quale valore è associato al simbolo “terzaLabel”?

Esercizio 4 – soluzione

Dato il seguente programma

```
.data 0x10010000
```

```
.text
```

```
#qui ci sono 2 istruzioni, nessuna pseudoistruzione e non ci sono definizioni o uso di label
```

```
j primaLabel
```

```
secondaLabel:
```

```
#qui ci sono 3 istruzioni, nessuna pseudoistruzione e non ci sono definizioni o uso di label
```

```
terzaLabel:
```

```
#qui ci sono 4 istruzioni, nessuna pseudoistruzione e non ci sono definizioni o uso di label
```

```
j secondaLabel
```

L'assemblatore assocerà correttamente un indirizzo a 2 label, ovvero “secondaLabel” e “terzaLabel”

Sarà invece “unresolved” dopo l'assemblaggio l'etichetta “primaLabel” che viene usata ma non dichiarata

Al simbolo “terzaLabel” verrà associato il valore $0x00400030 + 12_{10} (=3*4) \rightarrow 0x0040003c$ poichè si trova 3 istruzioni (ciascuna della dimensione di una word) dopo secondaLabel

Esercizio 5

- Qual è l'ordine corretto di esecuzione dei passi necessari per trasformare un programma scritto in un linguaggio di alto livello in un programma eseguibile?
 - Caricare, compilare, assemblare e linkare
 - Compilare, assemblare, linkare e caricare
 - Compilare, assemblare e linkare
 - Assemblare, compilare, linkare e caricare
- Qual è l'ordine corretto di esecuzione dei passi necessari per trasformare un programma scritto in un linguaggio di alto livello in un programma in esecuzione su un calcolatore?
 - Caricare, compilare, assemblare e linkare
 - Compilare, assemblare, linkare e caricare
 - Compilare, assemblare e linkare
 - Assemblare, compilare, linkare e caricare
- e per ottenere un file oggetto?
 - Caricare, compilare, assemblare e linkare
 - Compilare, assemblare e linkare
 - Compilare, assemblare, linkare e caricare
 - Compilare e assemblare

Esercizio 5 – soluzione

- Qual è l'ordine corretto di esecuzione dei passi necessari per trasformare un programma scritto in un linguaggio di alto livello in un programma eseguibile?
 - Caricare, compilare, assemblare e linkare
 - Compilare, assemblare, linkare e caricare
 - **Compilare, assemblare e linkare**
 - Assemblare, compilare, linkare e caricare
- Qual è l'ordine corretto di esecuzione dei passi necessari per trasformare un programma scritto in un linguaggio di alto livello in un programma in esecuzione su un calcolatore?
 - Caricare, compilare, assemblare e linkare
 - **Compilare, assemblare, linkare e caricare**
 - Compilare, assemblare e linkare
 - Assemblare, compilare, linkare e caricare
- e per ottenere un file oggetto?
 - Caricare, compilare, assemblare e linkare
 - Compilare, assemblare e linkare
 - Compilare, assemblare, linkare e caricare
 - **Compilare e assemblare**

Esercizio 6

- Quali informazioni sono output di un assemblatore?
- Quali informazioni sono output di un linker?

Esercizio 6 – soluzione

- L'output di un assemblatore è il file oggetto (**object file**), il cui contenuto è (da pag. A-13):
 - **object file header** – dimensione e posizione delle altre parti del file
 - **text segment** – codice macchina delle istruzioni che compongono il programma contenuto nel file (alcune potrebbero far riferimento a riferimenti non risolti - unresolved references)
 - **data segment** – rappresentazione binaria dei dati dichiarati nel file
 - **relocation information** – istruzioni e dati che dipendono da riferimenti assoluti
 - **symbol table** – associa le etichette agli indirizzi delle istruzioni cui fanno riferimento e elenca i riferimenti non risolti (unresolved references)
 - **debugging information** – descrizione sintetica del programma (usabile, ad esempio, da un programma di debugging)
- L'output di un linker è il programma eseguibile (**executable file**). Un programma eseguibile ha lo **stesso formato del file oggetto MA non contiene riferimenti non risolti e relocation information**

Esercizio 7

- Due procedure (A e B) sono state assemblate separatamente e i file oggetto risultanti sono linkati mettendo A prima di B
- Sapendo che:
 - la dimensione del testo di A è di 0x200 byte e dei suoi dati di 0x10 byte
 - la dimensione del testo di B è di 0x100 byte e dei suoi dati di 0x30 byte
- Qual è l'indirizzo base a partire dal quale sarà allocato il testo di A nel file eseguibile?
- Qual è, nel file oggetto di A, il valore del campo address di una chiamata alla procedura B (jal B)?
- Qual è, nel file eseguibile, il valore del campo address di una chiamata in A alla procedura B?

Esercizio 7 – soluzione

- Due procedure (A e B) sono state assemblate separatamente e i file oggetto risultanti sono linkati mettendo A prima di B
- Sapendo che:
 - la dimensione del testo di A è di 0x200 byte e dei suoi dati di 0x10 byte
 - la dimensione del testo di B è di 0x100 byte e dei suoi dati di 0x30 byte

- Qual è l'indirizzo base a partire dal quale sarà allocato il testo di A nel file eseguibile?

0x0040 0000 - all'inizio del text segment, subito dopo area riservata (v. pag A-20)

- Qual è, nel file oggetto di A, il valore del campo address di una chiamata alla procedura B (jal B)?

indefinito, perché sarà noto e inserito, solo quando A e B verranno uniti dal linker nell'eseguibile

- Qual è, nel file eseguibile, il valore del campo address di una chiamata in A alla procedura B?

Sapendo che B si trova dopo A e che A occupa uno spazio di 0x200 byte, B inizierà all'indirizzo

0x0040 0200

Esercizio 8

- Cosa deve fare un assemblatore a due passi quando durante il primo passo:
 - incontra un'istruzione preceduta da un'etichetta?
E.g. etichetta: add \$t0, \$t1, \$t2
 - incontra un'istruzione non preceduta da un'etichetta in cui è presente l'uso di un simbolo (etichetta risolta)?
E.g. j pippo
 - incontra un'istruzione senza etichetta in cui non sono presenti dei simboli?

Esercizio 8 – soluzione

- Cosa deve fare un assembler a due passi quando durante il primo passo:

- incontra un'istruzione preceduta da un'etichetta?

E.g. etichetta: add \$t0, \$t1, \$t2

scrive l'etichetta e il corrispondente valore nella tabella dei simboli

- incontra un'istruzione non preceduta da un'etichetta in cui è presente l'uso di un simbolo (etichetta risolta)?

E.g. j pippo

nulla perchè è nella seconda lettura del codice (istruzione per istruzione) che verrà cercato il simbolo nella tabella dei simboli e sostituito con il valore dell'indirizzo corrispondente, se presente

- incontra un'istruzione senza etichetta in cui non sono presenti dei simboli?

nulla

Esercizio 9

- Cosa è necessario fare per ottenere un programma eseguibile se le istruzioni che compongono il programma sono scritte in diversi file?

Esercizio 9 – soluzione

Cosa è necessario fare per ottenere un programma eseguibile se le istruzioni che compongono il programma sono scritte in diversi file?

- Oltre ad assemblare i file separatamente, serve fare tutto ciò che fa il linker, cioè:
 - Ricercare le librerie usate dal programma (e.g. syscall)
 - Determinare (leggendolo dall'object header) la porzione di memoria che dovrà essere occupata dal codice di ogni modulo e riposizionare le istruzioni di ogni modulo sistemando gli eventuali riferimenti assoluti
 - Risolvere i riferimenti tra file
 - ...

Esercizio 10

Quando un riferimento non risolto (“unresolved reference”) può indicare un errore del programmatore?

- Sempre
- Se non è risolto dopo che è stato assemblato il file che contiene l’uso del simbolo
- Se non è risolto dopo che sono stati assemblati tutti i file che compongono il programma
- Se non è risolto dopo che il linker ha terminato la sua attività

Esercizio 10 – soluzione

Quando un riferimento non risolto (“unresolved reference”) può indicare un errore del programmatore?

- Sempre
- Se non è risolta dopo che è stato assemblato il file che contiene l’uso del simbolo
- Se non è risolta dopo che sono stati assemblati tutti i file che compongono il programma
- **Se non è risolta dopo che il linker ha terminato la sua attività**

Esercizio 11

Se un programma è scritto in due file, quando un simbolo dichiarato come global in un file risolve un riferimento utilizzato in un altro file?

Esercizio 11 – soluzione

Se un programma è scritto in due file, quando un simbolo dichiarato come global in un file risolve un riferimento utilizzato in un altro file?

Quando entrambi fanno riferimento ad un'etichetta (simbolo) con lo stesso nome